

1 Problem Setup: DAE Approach

Consider a cylinder with radius of R , rolls without slip down a ramp with a slope of γ . The center of mass of the cylinder has been offset from the center (C) by a distance $d < R$. The total mass of the cylinder is M and it has a moment of inertia I^G about it's center of mass G .

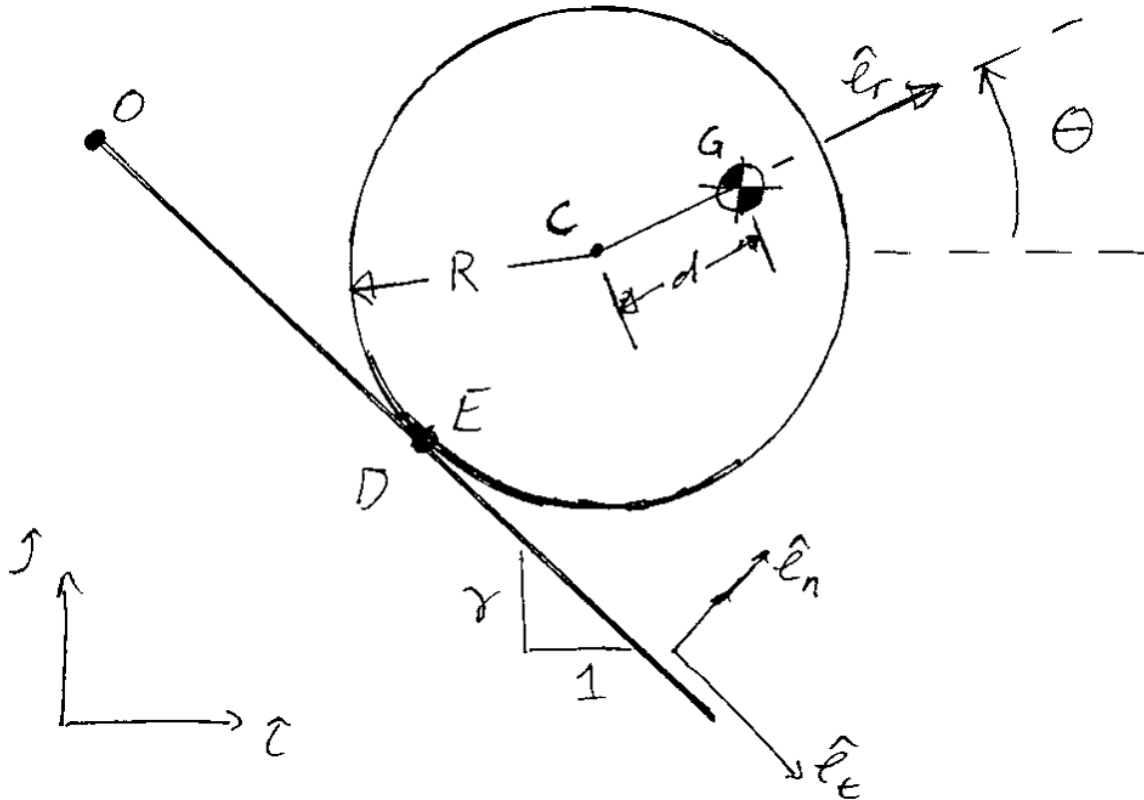


Figure 1: Diagram of rolling eccentric cylinder problem

As shown in Figure 1, there are three main reference frames for this problem. The inertial frame defined by $\hat{i}$, $\hat{j}$ and $\hat{k}$. The rotated inertial frame defined by $\hat{e}_n$ (normal to the ramp), $\hat{e}_t$ (tangent to the ramp) and $\hat{k}$.

$$\gamma_r = \tan^{-1}(\gamma) \quad (1)$$

$$\hat{e}_n = \sin(\gamma_r)\hat{i} + \cos(\gamma_r)\hat{j} \quad (2)$$

$$\hat{e}_t = \hat{e}_n \times \hat{k} \quad (3)$$

And finally the rotating frame fixed to the center of the cylinder defined by $\hat{e}_r$ (Points to COM), $\hat{e}_\theta$ and $\hat{k}$.

$$\hat{e}_n = \cos(\theta)\hat{i} + \sin(\theta)\hat{j} \quad (4)$$

$$\hat{e}_\theta = \hat{k} \times \hat{e}_r \quad (5)$$

We can now define the locations of the major points of interest for the problem in terms of these reference frames (Table 1).

Item of Interest	Definition
Center of the Cylinder (C)	$\vec{r}_{C/O} = x\hat{i} + y\hat{j}$
Contact Point (I)	$\vec{r}_{I/C} = -R\hat{e}_n$
Center of Mass (G)	$\vec{r}_{G/C} = -d\hat{e}_r$
G in Fixed Frame	$\vec{r}_{G/O} = \vec{r}_{G/C} + \vec{r}_{C/O}$
E relative to G	$\vec{r}_{E/G} = \vec{r}_{E/C} - \vec{r}_{G/C}$

Table 1: Positions of items of interest for the problem

Finally, we can define the contact force acting at E as $\vec{F} = F_n\hat{e}_n + F_t\hat{e}_t$, as shown in Figure 2.

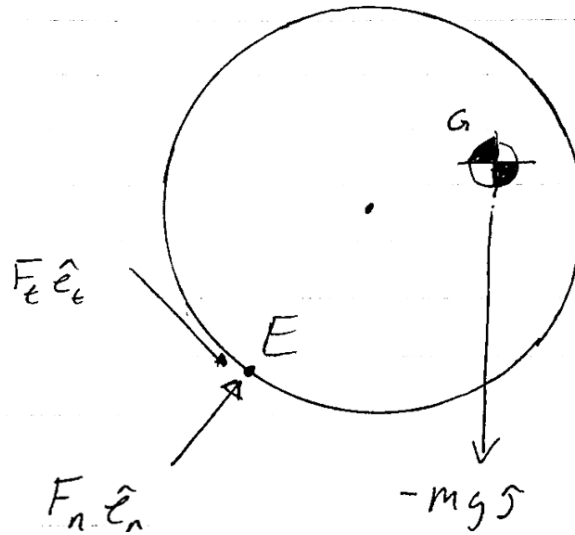


Figure 2: FBD for the Eccentric Cylinder

2 Equations of Motion

We can now solve this system using Differential Algebraical Expression to solve for the 5 unknowns of the system: $\ddot{x}$, $\ddot{y}$, $\ddot{\theta}$, F_n and F_t . The angular momentum balance about G (Eq. 6) and the linear momentum balance for the cylinder (Eq. 7), gives three of the five equations need for the DAE method.

$$\vec{r}_{I/G} \times \vec{F} = I^G \ddot{\theta} \hat{k} \quad (6)$$

$$M \ddot{\vec{r}}_{G/O} = -Mg \cdot \hat{j} + \vec{F} \quad (7)$$

The final two constraints on the systems is that cylinder shall not penetrate the ramp (Eq 8) and the cylinder shall role without slip (Eq. 9), which once differentiated give the final two equations (Eq 10 and 11).

$$\vec{r}_{C/O} \cdot \hat{e}_n = R \quad (8)$$

$$\dot{\vec{r}}_{C/O} \cdot \hat{e}_t = -R\dot{\theta}\hat{e}_t \quad (9)$$

$$\ddot{\vec{r}}_{C/O} \cdot \hat{e}_n = 0 \quad (10)$$

$$\ddot{\vec{r}}_{C/O} \cdot \hat{e}_t = -R\ddot{\theta}\hat{e}_t \quad (11)$$

Using MATLAB to handle the differentiation and rearrangement of terms gives the following equation of motion for the system:

$$x = A^{-1}b \quad (12)$$

$$A = \begin{bmatrix} M & 0 & -Md \sin(\theta) & -\hat{e}_n \cdot \hat{i} & -\hat{e}_t \cdot \hat{i} \\ 0 & M & -Md \cos(\theta) & -\hat{e}_n \cdot \hat{j} & -\hat{e}_t \cdot \hat{j} \\ 0 & 0 & -I & \vec{r}_{E/G} \times \hat{e}_n & \vec{r}_{E/G} \times \hat{e}_t \\ 1 & 0 & R\hat{e}_t \cdot \hat{i} & 0 & 0 \\ 0 & 1 & 0 & R\hat{e}_t \cdot \hat{j} & 0 \end{bmatrix} \quad (13)$$

$$x = \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{\theta} \\ F_n \\ F_t \end{bmatrix} \quad (14)$$

$$b = \begin{bmatrix} \dot{\theta}^2 M d \cos(\theta) \\ -M (g - d\dot{\theta}^2 \sin(\theta)) \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (15)$$

3 Numerical Simulation

Using the derived equations of motion (Eq 12), the cylinder was simulated using ode45 from rest, through the point of lift off. For the purposes of this simulation, lift-off was assumed to occur when F_n goes to zero, as the ramp can not exert a negative normal force. An ODE45 Event was used to monitor the normal force (F_n) and configured to stop the simulation once $F_n < 0$.

For the simulation and the rest of the present analysis the following parameters were used.

Parameter	Value
I^G	15
M	20
R	1
d	0.4
g	10
γ	1
$\theta(0)$	0
RelTol	1×10^{-12}
AbsTol	1×10^{-12}

Table 2: Parameters used for Simulation

4 Sanity Checks

4.1 Conservation of Energy

The kinetic energy of the system is defined as:

$$E_k = \frac{1}{2} I^G \dot{\theta}^2 + \frac{M}{2} \left| \dot{\vec{r}}_{G/O} \right|^2 \quad (16)$$

The potential energy was defined based on the height of the center of mass (G), as this results in the shortest possible formula.

$$E_p = Mg \cdot (\vec{r}_{G/O} \cdot \hat{j}) \quad (17)$$

As this system is conservative, the total energy of the system should remain constant. By tracking Equation 18 the error in simulated total energy of the system can be monitored. As shown in Figure 3, ϵ_e was less than 1×10^{-8} for the duration of the simulation, confirming that energy was conserved by the system.

$$\epsilon_e = E_k + E_p - E_{k,o} - E_{p,o} \quad (18)$$

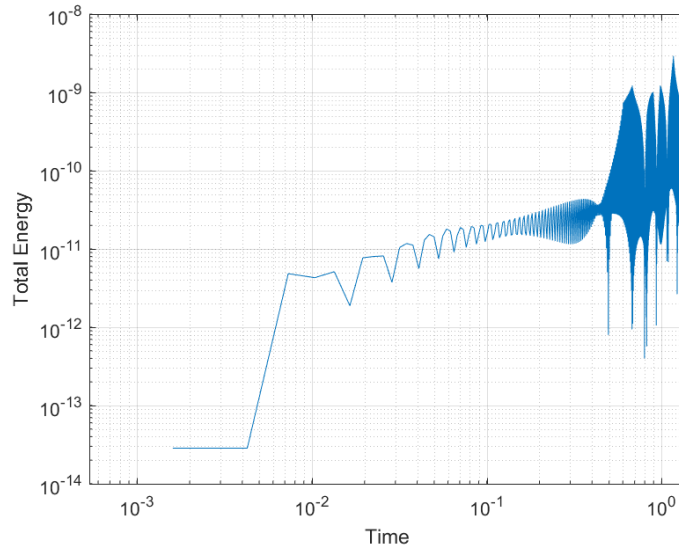


Figure 3: System Energy showing Conservation of Total Energy to within 5.9×10^{-9}

4.2 Constraint Violations

To confirm that the no slip (Equation 9) and no penetration (Equation 8) constraints were satisfied, the amount of error for both constraints was tracked over the course of the simulation, as shown in Figure 4.

For the no penetration constraint, the constraint error was defined using Equation 8. As shown in Figure 4, the absolute error remained below 10×10^{-14} for the course

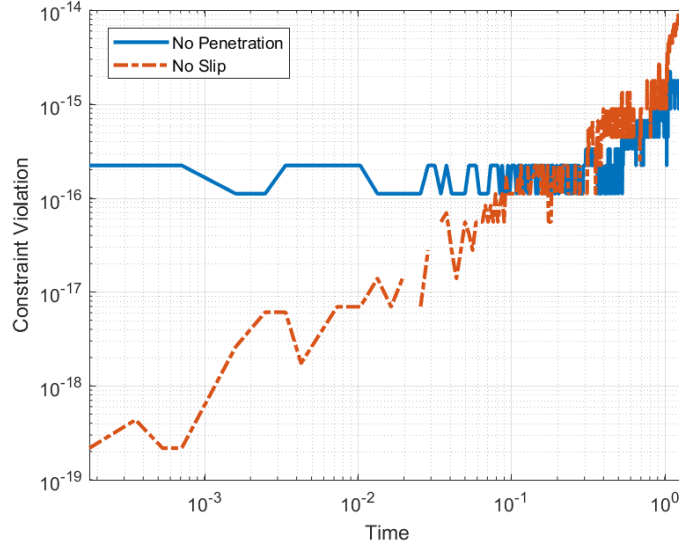


Figure 4: Plot of velocity relative to the slope showing no constraint violations prior to take off

of the simulation, confirming that the constraint was held. Similarly, the no slip constraint error was defined using Equation 20. While the no slip error grows on the order of $O(t)$, it still remains below 10×10^{-14} for the course of the simulation, confirming that the constraint was held.

$$\epsilon_p = \vec{r}_{C/O} \cdot \hat{e}_n - R \quad (19)$$

$$\epsilon_s = \dot{\vec{r}}_{C/O} \cdot \hat{e}_t + R\dot{\theta}\hat{e}_t \quad (20)$$

5 Coefficient of Friction

By taking the state information returned by ode45, and the DAE EOM (Eq 12), the constraint force can be computed for each time-step as shown in Figure 5. The largest calculated coefficient of friction was $\mu = 9.733$, however given the observed blow up at lift off a larger value of μ may be required.

$$\mu \geq \frac{F_t}{F_n} \quad (21)$$

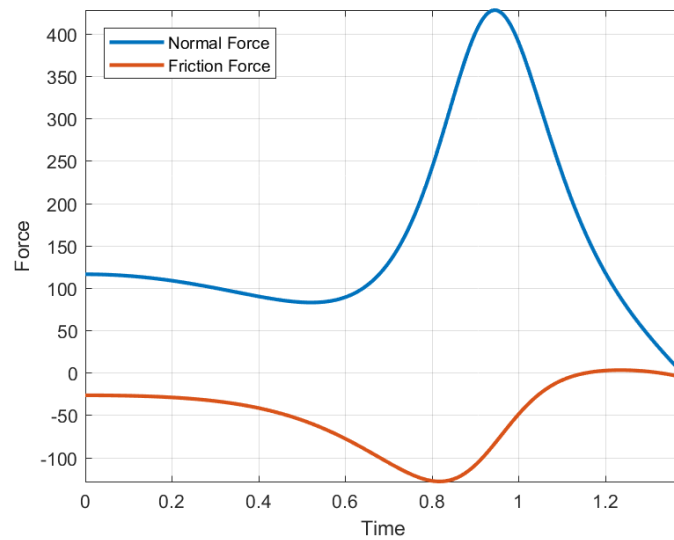


Figure 5: Constraint Force required for no slip motion

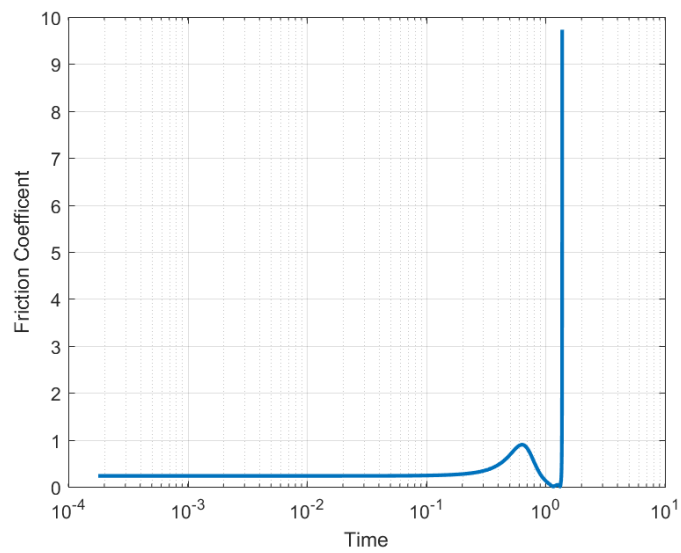


Figure 6: Required Friction Coefficient (μ) at each time-step

6 Time to Take Off

For the parameters used the cylinder rolled until $t = 1.3767$, before taking off. This was determined by using events to integrate until the normal constraint force became negative. As the ramp can only exert a positive normal force on the cylinder, a negative constraint force indicates that the cylinder was accelerating away from the map. That is the cylinder has taken off and is no longer rolling down the ramp, and is instead flying through the air.

7 Code

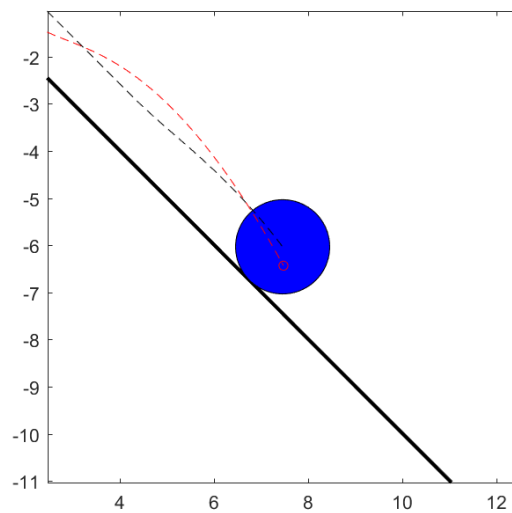


Figure 7: Cylinder touching down after briefly flying through the air

7.1 Derive DAE

```
clear variables
fprintf('Setting up system...')
syms x(t) y(t) theta(t) % State Variables of the Cylinder
syms R M I d gamma g mu real % Relevant Parameters
syms Fn Ft real % Contact Forces
```


Alexius Wadell: Rolling Eccentric Cylinder
MAE 5730 Intermediate Dynamics and Vibrations

```
% Fixed Frame
ih = [1 0 0]'; jh = [0 1 0]'; kh = [0 0 1]';

% Slope Frame
gammaRad = atan(gamma);
en = [sin(gammaRad); cos(gammaRad); 0]; % Vector Normal to Slope Surface
et = cross(en, kh); % Vector Tangent to Slope Pointing Down

% Cylinder Fixed Frame
er = [cos(theta); sin(theta); 0]; % Vector Rotating with Cylinder points at COM

% Positions of Points on Cylinder
CrelO = [x;y;0]; % Center of Cylinder in Fixed Frame
IrelC = -R*en; % Contact Point Relative to Center of Cylinder
GrelC = d*er; % Center of Mass Relative to Center of Cylinder
GrelO = GrelC + CrelO; % COM relative to Origin
IrelO = IrelC + CrelO; % Contact Point relative to Origin
IrelG = IrelC - GrelC; % Contact Point relative to COM

% Contact Force acting at C
F = Fn*en + Ft*et;

% LMB for Cylinder
LMB = M*diff(GrelO,2) == -M*g*jh + F;

% AMB about COM
AMB = cross(IrelG, F) == I*diff(theta,2)*kh;

%% Rolling Constraints
c1 = diff(CrelO) == -R*diff(theta)*et; % No Slip
c1 = diff(c1);

fprintf('done\n')
%% Derive EOM for Rolling
fprintf('Finding Rolling EOM...')
% Replace diff terms with variables
EOM = [ih'*LMB; jh'*LMB; kh'*AMB; ih'*c1; jh'*c1];
[newEq, newVar] = reduceOrder(EOM, {x, y, theta});

% Solve for accelerations and constraint forces
ddState = [newVar(:,3); Fn; Ft];
[A, b] = equationsToMatrix(newEq, ddState);
A = simplify(A); b = simplify(b);

matlabFunction(A, b, 'File', 'eom_rolling');
```

```
fprintf('done\n')

%% Derive EOM for Ballistic
fprintf('Finding Ballistic EOM...')
% LMB and AMB for when ballistic
LMB = M*diff(GrelO,2) == -M*g*jh;
AMB = 0 == I*diff(theta,2)*kh;

% Replace diff terms with variables
EOM = [ih'*LMB; jh'*LMB; kh'*AMB];
vars = {x, y, theta};
[newEq, newVar] = reduceOrder(EOM, vars);

% Solve for accelerations
ddState = newVar(:,3);
[A, b] = equationsToMatrix(newEq, ddState);
A = simplify(A); b = simplify(b);

matlabFunction(A, b, 'File', 'eom_ballistic');
fprintf('done\n')
%% Compute IC from Wo
fprintf('Finding IC given initial omega...')
% Cylinder starts with the contact point at the origin with an initial
% angular velocity w0
C0 = -IrelC;
v0 = -R*diff(theta)*et;
z0 = [ih'*C0; jh'*C0; theta; ih'*v0; jh'*v0; diff(theta)];

z0 = reduceOrder(z0, {x,y,theta});
matlabFunction(z0, 'File', 'computeIC');
fprintf('done\n')

%% Conservation of Energy
fprintf('Finding system energy...')
v2 = sum(diff(GrelO).^2);
Ek = 0.5*(M*v2 + I*diff(theta)^2);
Ep = M*g*(jh'*GrelO);

Ek = reduceOrder(Ek, {x,y,theta});
Ep = reduceOrder(Ep, {x,y,theta});

matlabFunction(Ek, Ep, 'File', 'energy_con');
fprintf('done\n')

%% Constraint Violations
fprintf('Checking constraint violations...')
```

```
c1 = en'*Crel0 - R;          % No Penetration
c2 = et'*diff(Crel0) + R*diff(theta); % No Slip

c1 = reduceOrder(c1, {x,y,theta});
c2 = reduceOrder(c2, {x,y,theta});

matlabFunction(c1, c2, 'File', 'constraint-violations');
fprintf('done\n')
```

7.2 Analysis

```
clear variables; close all
% Set Parameters
p.I= 15;
p.M= 20;
p.R= 1;
p.d= 0.4;
p.g= 10;
p.gamma = 1;
p.theta0 = 0;
p.omega0 = 0;
default = p;

%% Simulate Base Case
[sim1, t, z] = simulate(p);
fprintf('Time to jump: %f\n', sim1.tlift);

%% Check for Energy Conservation
[Ek,Ep] = energy_con(sim1.dx, sim1.dy,...
    sim1.dtheta, p.I, p.M, p.d, p.g,...
    sim1.theta, sim1.y);
Et = Ek + Ep;
E_e = Et - Et(1);

figure; box on; grid on; hold on;
plot(sim1.t, abs(E_e))
xlim([0,sim1.tlift])
ax = gca; ax.XScale = 'log'; ax.YScale = 'log';
xlabel('Time'); ylabel('Total Energy');
saveas(gcf, 'img\energy_con', 'png')

%% Check for Constraint Violations
[c1,c2] = constraint-violations(...
    sim1.dx, sim1.dy, sim1.dtheta,...
    p.R, p.gamma,...
```

```
    sim1.x, sim1.y);
figure; box on; hold on; grid on;
plot(sim1.t, abs(c1), '-', 'LineWidth', 2)
plot(sim1.t, abs(c2), '-.', 'LineWidth', 2)
legend('No Penetration',...
       'No Slip',...
       'Location', 'northwest')
ax = gca; ax.XScale = 'log'; ax.YScale = 'log';
xlim([0,sim1.tlift])
xlabel('Time'); ylabel('Constraint Violation')
saveas(gcf, 'img\constraint_check', 'png')

%% Compute Constraint Force
if ~isnan(sim1.tlift)
    iLift = sum(t<sim1.tlift);
else
    iLift = length(sim1.t);
end
sim1.Fn = zeros(size(sim1.t)); sim1.Ft = sim1.Fn;
for i = iLift:-1:1
    [A,b] = eom_rolling(sim1.dtheta(i),p.I,p.M,p.R,p.d,p.g,p.gamma,sim1.theta(i));
    sol = A\b;
    sim1.Fn(i) = sol(4);    sim1.Ft(i) = sol(5);
end

% Compute Required Mu
sim1.mu = abs(sim1.Ft./sim1.Fn);

figure; box on; hold on; grid on;
plot(sim1.t, sim1.Fn, 'LineWidth', 2)
plot(sim1.t, sim1.Ft, 'LineWidth', 2)
legend('Normal Force', 'Friction Force', 'Location', 'northwest')
xlabel('Time'); ylabel('Force')
axis tight
xlim([0,sim1.tlift])
saveas(gcf, 'img\constaint_force', 'png')

figure; box on; hold on;
plot(sim1.t, sim1.mu, 'LineWidth', 2)
xlabel('Time'); ylabel('Friction Coefficient')
ax = gca; ax.XScale = 'log';
grid on;
saveas(gcf, 'img\friction_mu', 'png')

%% Animate
animate(t,z,sim1,p);
```

```
saveas(gcf, 'img\animate', 'png')
```

7.3 Simulate

```
function [data, t, z] =simulate(p)
% Find IC for system
z0 = computeIC(p.omega0, p.R, p.gamma, p.theta0);

% ODE Options
tol = 1e-12;
optsRoll = odeset('RelTol', tol, 'AbsTol', tol, 'Events', @hasJumped);
opsBallistic = odeset('RelTol', tol, 'AbsTol', tol, 'Events', @hasLanded);

%% Simulate Cylinder
tFinal = 40;
[tR, zR] = ode45(@RHS_rolling, [0, tFinal], z0, optsRoll, p);

if (tR(end) < tFinal)
    tlift = tR(end); z0n = zR(end,:);
    z0n(2) = z0(2) -p.gamma*(z0n(1) - z0(1)); %Ensure tangency
    [tB, zB] = ode45(@RHS_ballistic, [tlift, tFinal], z0n, opsBallistic, p);

    % Combine Results
    t = [tR(1:end-1); tB]; z = [zR(1:end-1,:); zB];
else
    tlift = nan;
    t = tR; z = zR;
end

terms = {'x', 'y', 'theta', 'dx', 'dy', 'dtheta'};
data = sim2struct(t, z, terms);
data.tlift = tlift;
end
```

7.4 RHS

```
function dz = RHS_rolling(t, z, p)
% Extract Parameters
I = p.I;
M = p.M;
R = p.R;
```

```
d = p.d;
g = p.g;
gamma = p.gamma;

% Extract State
x = z(1); y = z(2); theta = z(3);
D1_x = z(4); D1_y = z(5); D1_theta = z(6);

[A,b] = eom_rolling(D1_theta,I,M,R,d,g,gamma,theta);
sol = A\b;
D2_x = sol(1); D2_y = sol(2); D2_theta = sol(3);

dz = [D1_x; D1_y; D1_theta; D2_x; D2_y; D2_theta];
end
```

7.5 DAE Matrices

```
function [A,b] = eom_rolling(D1_theta,I,M,R,d,g,gamma,theta)
%EOM_ROLLING
% [A,B] = EOM_ROLLING(D1_THETA,I,M,R,D,G,GAMMA,THETA)

% This function was generated by the Symbolic Math Toolbox version 7.2.
% 10-Dec-2017 15:08:58

t2 = gamma.^2;
t3 = t2+1.0;
t4 = 1.0./sqrt(t3);
t5 = cos(theta);
t6 = sin(theta);
t7 = R.*t4;
t8 = R.*gamma.*t4;
A = reshape([M,0.0,0.0,1.0,0.0,0.0,M,0.0,0.0,1.0,-M.*d.*t6,M.*d.*t5,-I,t7,-t8,-gamma.*t
if nargin > 1
    t9 = D1_theta.^2;
    b = [M.*d.*t5.*t9;-M.*(g-d.*t6.*t9);0.0;0.0;0.0];
end
```

7.6 Energy Conservation

```
function [Ek,Ep] = energy_con(D1_x,D1_y,D1_theta,I,M,d,g,theta,y)
%ENERGY_CON
% [EK,EP] = ENERGY_CON(D1_X,D1_Y,D1_THETA,I,M,D,G,THETA,Y)
```

```
% This function was generated by the Symbolic Math Toolbox version 7.2.
% 10-Dec-2017 15:09:12

t2 = D1_y+D1_theta.*d.*cos(theta);
t4 = sin(theta);
t3 = D1_x-D1_theta.*d.*t4;
Ek = D1_theta.^2.*I.*(1.0./2.0)+M.*(t2.^2+t3.^2).*(1.0./2.0);
if nargout > 1
    Ep = M.*g.*(y+d.*t4);
end
```

7.7 Check Constraints

```
function [c1,c2] = constraint_violations(D1_x,D1_y,D1_theta,R,gamma,x,y)
%CONSTRAINT_VIOLATIONS
% [C1,C2] = CONSTRAINT_VIOLATIONS(D1_X,D1_Y,D1_THETA,R,GAMMA,X,Y)

% This function was generated by the Symbolic Math Toolbox version 7.2.
% 10-Dec-2017 15:09:15

t2 = gamma.^2;
t3 = t2+1.0;
t4 = 1.0./sqrt(t3);
c1 = -R+t4.*y+gamma.*t4.*x;
if nargout > 1
    c2 = D1_x.*t4+D1_theta.*R-D1_y.*gamma.*t4;
end
```